

Guardrails in LLM / Agentic Systems — Concepts and Python Implementation

1. What Are Guardrails?

In the context of LLM-powered and agentic systems, **guardrails** are a set of programmatic checks, constraints, and correction mechanisms that sit around a model's input and output to ensure the system behaves safely, reliably, and within specification — regardless of what the underlying LLM produces.

An LLM is a probabilistic, non-deterministic component. Guardrails convert this into a system with **deterministic safety and correctness boundaries**, by validating:

- **What goes into the model** (input guardrails)
- **What comes out of the model** (output guardrails)
- **What the model is allowed to do** (behavioral / action guardrails), especially in agentic or tool-using systems

Formally, a guardrail can be viewed as a function

$$G : \quad \rightarrow \{ \text{pass, fail, correct(')} \}$$

applied either to the input space *in* before the LLM call, or the output space *out* after it, where a failure can either **block**, **retry**, or **rewrite** the offending content.

2. Why Guardrails Matter

Risk without guardrails	Consequence
Prompt injection / jailbreaks	Model executes unintended instructions
Hallucinated facts or entities	Downstream systems act on false data
PII / sensitive data leakage	Compliance and privacy violations
Malformed structured output (JSON/YAML)	Pipeline crashes when parsing
Off-topic or toxic generations	Brand and safety risk
Unbounded tool/agent actions	Unsafe or irreversible side effects (e.g. an agent calling a delete API)

This is especially relevant in **multi-agent, tool-augmented pipelines** (e.g. an enrichment worker that calls DB tools and a web tool, then a reflection agent, then a synthesizer) —

guardrails are what keep each hop's output trustworthy enough to feed into the next agent.

3. Taxonomy of Guardrails

3.1 Input Guardrails

Applied to the user/system prompt *before* it reaches the LLM.

- Prompt injection detection
- PII redaction/masking
- Topic/scope classifiers (is this query in-domain?)
- Rate limiting / length limits
- Toxicity / jailbreak pattern filters

3.2 Output Guardrails

Applied to the LLM's response *before* it is returned or passed downstream.

- **Structural validation** — valid JSON/YAML/XML, matches a schema
- **Semantic validation** — factual grounding, no hallucinated entities, confidence thresholds
- **Content safety** — toxicity, bias, PII in output
- **Format constraints** — regex patterns, enum membership, numeric ranges
- **Self-consistency checks** — e.g. does the synthesizer's chosen value actually appear in the enrichment worker's collected evidence?

3.3 Behavioral / Action Guardrails (Agentic)

Applied to **decisions and tool calls**, not just text.

- Tool allow-lists (which tools an agent may invoke)
- Argument validation before tool execution (e.g. don't allow a DELETE call)
- Human-in-the-loop approval gates for irreversible actions
- Max iteration / recursion limits (prevent infinite agent loops)
- Cost/latency budgets per run

3.4 Correction Strategies

When a guardrail fails, the system typically does one of:

1. **Reject** — return an error / refuse
2. **Retry** — re-prompt the LLM with the validation error appended (self-healing)
3. **Fallback** — return a default/safe value
4. **Reroute** — send back to a different node (e.g., in LangGraph, route back to the enrichment worker with hints — which is exactly what your reflection agent already does conceptually)

4. Implementation Approaches in Python

There are three common tiers of implementation, from lightweight to comprehensive:

1. **Custom guardrails** with `pydantic` + `regex` — fast, dependency-light, full control
2. `guardrails-ai` library — declarative validators, retry/re-ask built in
3. **LLM-as-judge guardrails** — a second (often smaller) model validates the first model's output semantically

We'll implement all three, then show how they compose inside a LangGraph node.

4.1 Environment Setup

```
bash
```

```
pip install pydantic guardrails-ai openai langgraph --quiet
```

4.2 Tier 1 — Custom Guardrails with Pydantic

Pydantic gives you strong **structural + type-level validation** with minimal overhead — ideal for enforcing schemas on structured LLM output (e.g. an enriched attribute record).

```
python
```

```

from pydantic import BaseModel, field_validator, ValidationError
from typing import Optional
import re

class EnrichedAttribute(BaseModel):
    attribute_name: str
    value: str
    confidence: float
    source: str

    @field_validator("confidence")
    @classmethod
    def confidence_in_range(cls, v: float) -> float:
        if not (0.0 <= v <= 1.0):
            raise ValueError("confidence must be between 0 and 1")
        return v

    @field_validator("value")
    @classmethod
    def value_not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("value cannot be empty")
        return v

    @field_validator("source")
    @classmethod
    def source_is_known(cls, v: str) -> str:
        allowed = {"db", "web", "llm_inference"}
        if v not in allowed:
            raise ValueError(f"source must be one of {allowed}")
        return v

def validate_llm_output(raw_json: dict) -> Optional[EnrichedAttribute]:
    """Guardrail: structural + semantic validation of LLM output."""
    try:
        return EnrichedAttribute(**raw_json)
    except ValidationError as e:
        print(f"[GUARDRAIL FAILED] {e}")
        return None

```

A simple **PII redaction guardrail** using regex (a common input/output guardrail):

python

```
PII_PATTERNS = {
    "email": re.compile(r"[\w\.-]+@[\w\.-]+\.\w+"),
    "phone": re.compile(r"\b\d{3}[-.\s]?d{3}[-.\s]?d{4}\b"),
    "ssn": re.compile(r"\b\d{3}-d{2}-d{4}\b"),
}

def redact_pii(text: str) -> str:
    for label, pattern in PII_PATTERNS.items():
        text = pattern.sub(f"[REDACTED_{label.upper()}]", text)
    return text
```

4.3 Tier 2 — `guardrails-ai` (Declarative Validators + Auto-Retry)

`guardrails-ai` wraps an LLM call with a **RAIL spec** (or a Pydantic model) and automatically re-prompts the model on failure, which is very useful when you want the LLM to *self-correct* rather than simply reject.

python

```

import guardrails as gd
from pydantic import BaseModel, Field
from guardrails.hub import ValidRange, ValidChoices

class AttributeExtraction(BaseModel):
    attribute_name: str
    value: str = Field(description="Extracted value from source document")
    confidence: float = Field(validators=[ValidRange(min=0, max=1)])
    source: str = Field(validators=[ValidChoices(choices=["db", "web", "llm_inf

guard = gd.Guard.from_pydantic(
    output_class=AttributeExtraction,
    prompt=(
        "Extract the attribute '{attribute_name}' from the following text.\n"
        "Text: {document_text}\n"
        "Return valid JSON matching the schema."
    ),
)

raw_llm_response, validated_output, *_ = guard(
    llm_api=lambda prompt, **kw: call_your_llm(prompt), # your LLM wrapper
    prompt_params={"attribute_name": "annual_turnover", "document_text": "..."},
    num_reasks=2, # auto re-prompt on validation failure
)

print(validated_output)

```

Key benefit over Tier 1: on failure, `guardrails-ai` **automatically re-asks the LLM**, injecting the validation error into the follow-up prompt — a built-in version of the retry/reroute pattern.

4.4 Tier 3 — LLM-as-Judge (Semantic Guardrails)

Structural validators can't catch hallucination or factual drift. For that, use a second, cheaper LLM call as a **judge**, comparing the output against the evidence it was derived from.

python

```
JUDGE_PROMPT = """You are a strict fact-checking guardrail.

Claim: {claim}
Evidence: {evidence}

Does the evidence support the claim? Respond with ONLY one word: SUPPORTED or UNSUPPORTED"""

def factual_guardrail(claim: str, evidence: str, llm_call) -> bool:
    verdict = llm_call(JUDGE_PROMPT.format(claim=claim, evidence=evidence))
    return verdict.strip().upper() == "SUPPORTED"

# Example usage inside a synthesizer step
if not factual_guardrail(
    claim=f"annual_turnover = {synthesized_value}",
    evidence=collected_evidence_text,
    llm_call=call_your_llm,
):
    # Guardrail failed -> reroute to reflection agent instead of finalizing
    route_to = "reflection_agent"
```

5. Composing Guardrails as a Pipeline

In practice you chain guardrails into a single validation pipeline, short-circuiting on the first failure:

```
python
```

```

from dataclasses import dataclass
from typing import Callable, List, Tuple

@dataclass
class GuardrailResult:
    passed: bool
    reason: str = ""

def run_guardrail_pipeline(
    output: dict,
    guardrails: List[Callable[[dict], GuardrailResult]],
) -> Tuple[bool, str]:
    for guardrail in guardrails:
        result = guardrail(output)
        if not result.passed:
            return False, result.reason
    return True, "all guardrails passed"

def structural_check(output: dict) -> GuardrailResult:
    try:
        EnrichedAttribute(**output)
        return GuardrailResult(True)
    except ValidationError as e:
        return GuardrailResult(False, str(e))

def pii_check(output: dict) -> GuardrailResult:
    cleaned = redact_pii(output.get("value", ""))
    if cleaned != output.get("value", ""):
        return GuardrailResult(False, "PII detected in value field")
    return GuardrailResult(True)

def confidence_threshold_check(output: dict, min_conf: float = 0.6) -> GuardrailResult:
    if output.get("confidence", 0) < min_conf:
        return GuardrailResult(False, f"confidence below threshold {min_conf}")
    return GuardrailResult(True)

passed, reason = run_guardrail_pipeline(
    output=llm_output_dict,
    guardrails=[structural_check, pii_check, confidence_threshold_check],
)

```


6. Guardrails Inside a LangGraph Agentic Workflow

This maps directly onto an architecture like `(enrichment_worker → reflection_agent → synthesizer)`: guardrails become a **conditional edge** that routes based on validation outcome rather than always advancing linearly.

```
python
```

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class EnrichmentState(TypedDict):
    attribute_name: str
    collected_evidence: str
    candidate_value: dict
    retries: int

def guardrail_node(state: EnrichmentState) -> EnrichmentState:
    passed, reason = run_guardrail_pipeline(
        state["candidate_value"],
        guardrails=[structural_check, pii_check, confidence_threshold_check],
    )
    state["guardrail_passed"] = passed
    state["guardrail_reason"] = reason
    return state

def route_after_guardrail(state: EnrichmentState) -> str:
    if state["guardrail_passed"]:
        return "synthesizer"
    if state["retries"] >= 3:
        return END # circuit breaker: stop retrying indefinitely
    return "reflection_agent" # reroute for another enrichment pass

graph = StateGraph(EnrichmentState)
graph.add_node("guardrail_check", guardrail_node)
# ... add enrichment_worker, reflection_agent, synthesizer nodes ...
graph.add_conditional_edges(
    "guardrail_check",
    route_after_guardrail,
    {"synthesizer": "synthesizer", "reflection_agent": "reflection_agent", END:
}
```

The important design point: the guardrail node doesn't just pass/fail — it **feeds** `guardrail_reason` **back into the state**, so the reflection agent (or the re-ask prompt) has a concrete, actionable error message to correct against, rather than blindly retrying.

7. Best Practices

- 1. **Fail closed, not open** — on ambiguous validation, default to rejecting/rerouting rather than letting risky output through.
 - 2. **Bound your retries** — always cap re-ask/reroute loops (a `retries` counter with a circuit breaker, as above) to avoid infinite agent loops and runaway cost.
 - 3. **Layer guardrails by cost** — run cheap structural checks (regex, pydantic) before expensive semantic checks (LLM-as-judge), so you fail fast.
 - 4. **Log every guardrail failure** — validation failures are some of the highest-signal data for improving prompts and few-shot examples over time.
 - 5. **Keep guardrails decoupled from the LLM call itself** — treat them as pure functions on structured state so they're independently testable and reusable across agents (worker, reflection, synthesizer).
 - 6. **Version your schemas** — as attributes evolve (e.g., new enrichment fields), guardrail schemas should be versioned alongside your YAML business rules, not hardcoded ad hoc.
-

8. Summary

Layer	Tooling	Purpose
Structural	<code>pydantic</code> , regex	Type/format correctness
Declarative + self-healing	<code>guardrails-ai</code>	Schema validation with auto re-ask
Semantic	LLM-as-judge	Factual grounding, hallucination detection
Behavioral	Custom routing logic	Tool/action safety, agent loop control
Orchestration	LangGraph conditional edges	Wiring guardrails into multi-agent flow control

Guardrails are not a single library call — they're a **layered validation architecture** that turns a probabilistic LLM into a system with predictable failure modes, and in an agentic pipeline, they double as the routing logic that decides whether output is trustworthy enough to hand off to the next agent.